CS471 – Web Technologies (Laboratory)	Qassim University College of Computer	Lab 6
		HTML Form

This lab session covers the basics of HTML form. By the end of this session, students should have a very good understanding of how to build a basic form and how to Handle form in Django.

Pre-lab Preparation:

1. Having bookmodule and usermodule in apps directory, each containing separate URL configurations.
2. Template directory located in 'apps' with necessary settings.
3. Master template 'base.html'

Lab Activities: A complete webpage example with html form and Handling form in Django

Task 1: Build HTML form template

Step 1: Using the master template base.html from Lab4, import it in <head> section in base.html the following:

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-awesome.min.css">

{% block stylesheets %}{% endblock stylesheets %}

</head>

<body>
```

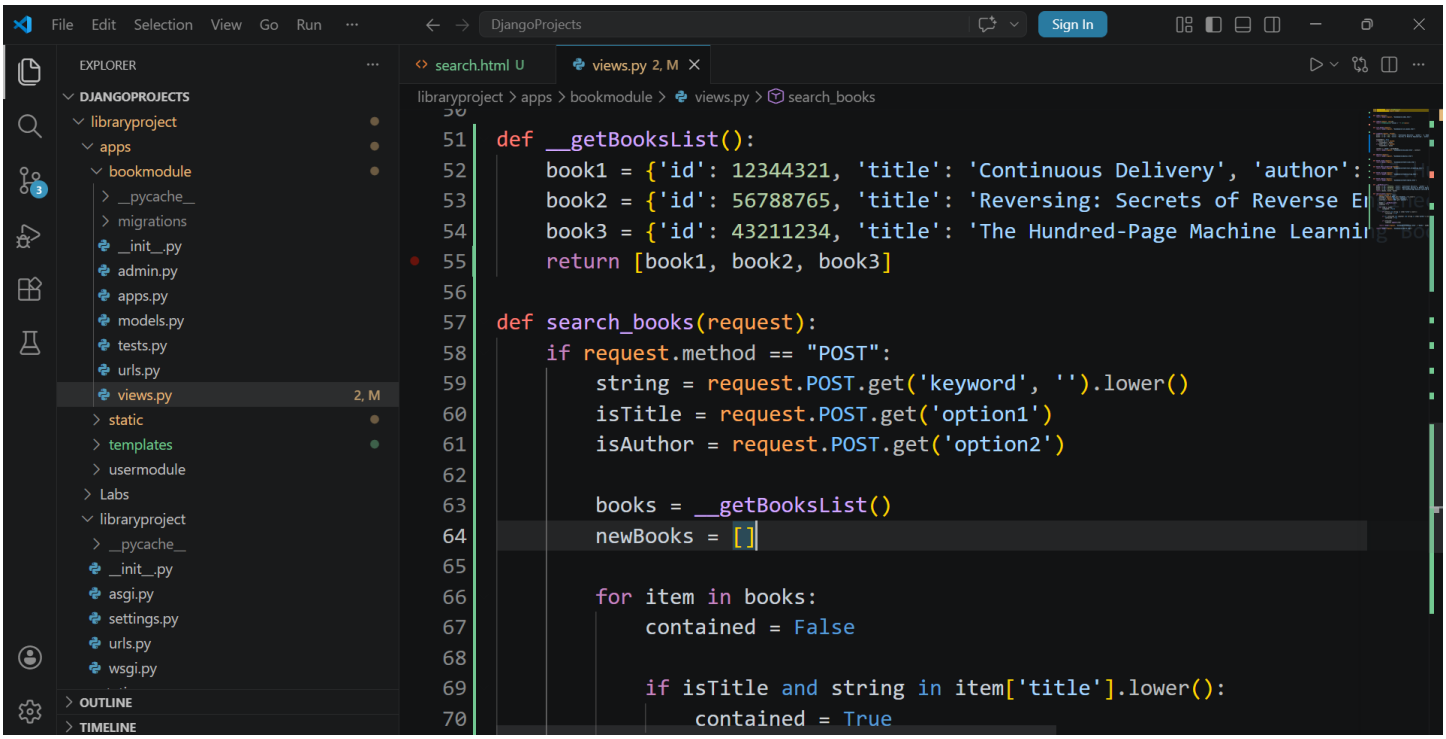
Step 2: using the master template 'base.html', Create a view function and URL '/books/search' to a page that contains the following HTML form:

```
{% extends "layouts/base.html" %}
{% load static %}
{% block title %} Searching Page {% endblock %}
{% block content %}
<form method="post"
    {% csrf_token %}

    <label>Keywords:</label>
    <input type="text" name="keyword" placeholder="enter a keyword"/>

    <br><br>

    <label>Search in:</label>

    <input type="checkbox" name="option1"/>
    <label>Title</label>

    <input type="checkbox" name="option2"/>
    <label>Author</label>

    <br><br>

    <button type="submit">Search</button>
</form>
{% endblock %}
```

Step 3: After running the server, open the web browser and navigate to http://127.0.0.1:8000/books/search. You should see the form.

Task 2: Handle the form once submitted

Step 1: In 'apps/bookmodule/view.py', create the following function:



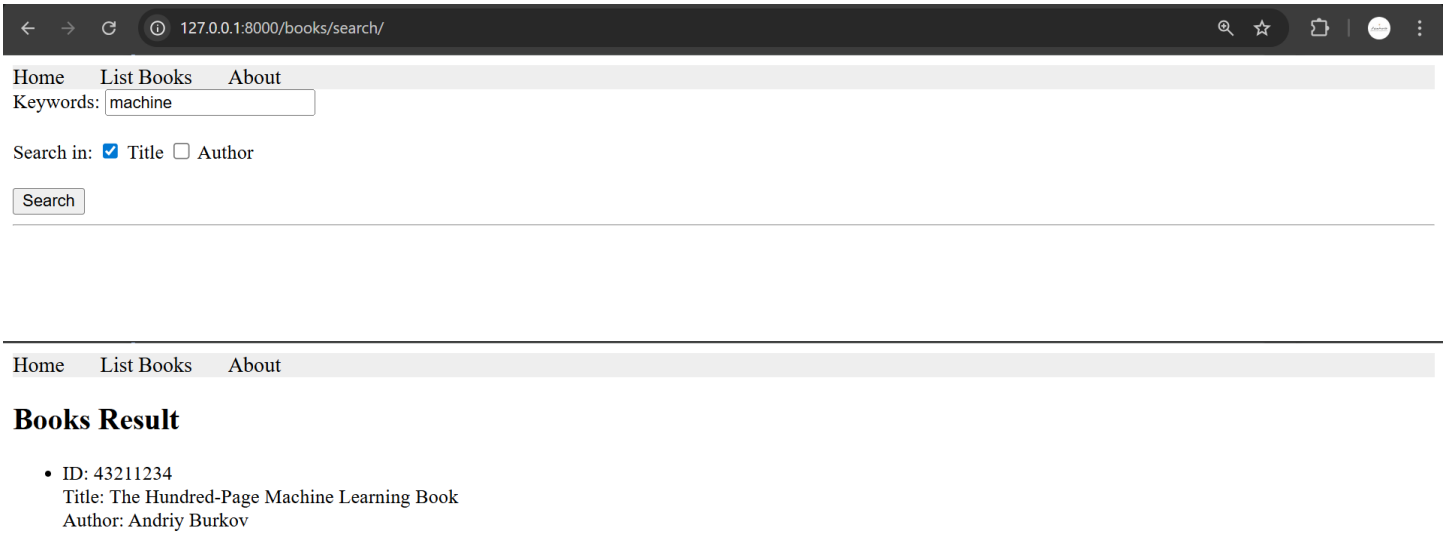
Step 2: In the same corresponding view/controller in views.py that shows the form, add the code where If there is an action in the form, the form needs to be handled in that view.
Note: it is possible to direct the action to an exclusive view function specific to handle the submitted form.

```python
def search_books(request):
    if request.method == "POST":
        string = request.POST.get('keyword').lower()
        isTitle = request.POST.get('option1')
        isAuthor = request.POST.get('option2')

        books = __getBooksList()
        newBooks = []

        for item in books:
            contained = False

            if isTitle and string in item['title'].lower():
                contained = True

            if not contained and isAuthor and string in item['author'].lower():
                contained = True

            if contained:
                newBooks.append(item)

        return render(request, 'bookmodule/bookList.html', {'books': newBooks})
```

Step 3: Create an HTMl file '`bookList.html`' to list the books (you should use the master template '`base.html`')

search.html U | views.py 2, M | bookList.html U | urls.py 1, M | base.html M

libraryproject > apps > templates > bookmodule > bookList.html > ul

```html
{% extends "layouts/base.html" %}
{% block title %} Books List {% endblock %}
{% block content %}

<h2>Books Result</h2>
<ul
{% for book in books %}

<li>

ID: {{book.id}} <br>
Title: {{book.title}} <br>
Author: {{book.author}}

</li>

<br>

{% endfor %}

</ul>

{% endblock %}
```

Step 4: Now from the web browser, navigate to http://127.0.0.1:8000/books/search, and submit the form with data.